Assignment 2 - Data Wrangling

Nafis

October 2026

Assignment 2 - Data Wrangling

1. Create a vector v with all integers 0-30, and a vector w with every third integer in the same range (Hint: you will need the `seq()` function).

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
v <- seq(0, 30)
w <- seq(0, 30, by = 3)
```

a) What is the difference in lengths of the vectors v and w? [0.25]

```
vw_diff <- length(v) - length(w)
vw_diff
```

```
## [1] 20
```

v has 31 elements and w has 11 elements, so the difference is 20.

b) Create a new vector, v_square, with the square of elements at indices 3, 6, 7, 10, 15, 22, 23, 24, and 30 from the variable v. Hint: use indexing. Calculate the mean and median of the first five values from v_square. [0.5]

```
v_square <- v[c(3, 6, 7, 10, 15, 22, 23, 24, 30)]^2
v_square
```

```
## [1]   4  25  36  81 196 441 484 529 841
```

```r
mean_v <- mean(v_square[1:5])
median_v <- median(v_square[1:5])
mean_v
```

```
## [1] 68.4
```

```r
median_v
```

```
## [1] 36
```

The mean of the first five values from v_square is 68.4 and the median of the first five values from v_square is 36.

c) Create a boolean vector v_bool, indicating which vector v elements are bigger than 20. How many values are over 20? Hint: In R, TRUE = 1, and FALSE = 0, so you can use simple arithmetic to find this out. [1]

```r
v_bool <- v > 20
sum(v_bool)
```

```
## [1] 10
```

```r
v[v_bool]
```

```
##  [1] 21 22 23 24 25 26 27 28 29 30
```

There are 10 values bigger than 20.

2. Use the `CO2` dataset that pre-exists in R. Use the code below to get information about this dataset.

```r
library(tidyverse)
?CO2
```

```
## starting httpd help server ... done
```

a) Create a summary table, grouped by the plant and the treatment, that includes the mean, standard deviation, and number of data rows. Is this dataset balanced (i.e., are there the same number of observations for each treatment)? [0.5]

```r
CO2 %>%
  group_by(Plant, Treatment) %>%
  summarize(
    mean = mean(uptake),
    stdev = sd(uptake),
    count = n()
  )
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Plant and Treatment.
## i Output is grouped by Plant.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Plant, Treatment))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
## # A tibble: 12 x 5
## # Groups:   Plant [12]
##    Plant Treatment   mean stdev count
##    <ord> <fct>      <dbl> <dbl> <int>
##  1 Qn1   nonchilled  33.2  8.21     7
##  2 Qn2   nonchilled  35.2 11.0      7
##  3 Qn3   nonchilled  37.6 10.3      7
##  4 Qc1   chilled     30.0  8.33     7
##  5 Qc3   chilled     32.6 10.3      7
##  6 Qc2   chilled     32.7 11.3      7
##  7 Mn3   nonchilled  24.1  6.48     7
##  8 Mn2   nonchilled  27.3  7.65     7
##  9 Mn1   nonchilled  26.4  8.69     7
## 10 Mc2   chilled     12.1  2.19     7
## 11 Mc3   chilled     17.3  3.05     7
## 12 Mc1   chilled     18    4.12     7
```

Yes, the dataset is balanced because every Plant-Treatment group has 7 observations.

b) Using the summarized table from part a), reorder the table from largest to smallest mean uptake. What plant and treatment has the highest and lowest mean uptake? [0.5]

```
CO2 %>%
  group_by(Plant, Treatment) %>%
  summarize(
    mean = mean(uptake),
    stdev = sd(uptake),
    count = n()
  ) %>%
  arrange(desc(mean))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Plant and Treatment.
## i Output is grouped by Plant.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Plant, Treatment))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
## # A tibble: 12 x 5
## # Groups:   Plant [12]
##    Plant Treatment   mean stdev count
##    <ord> <fct>      <dbl> <dbl> <int>
##  1 Qn3   nonchilled  37.6 10.3      7
##  2 Qn2   nonchilled  35.2 11.0      7
##  3 Qn1   nonchilled  33.2  8.21     7
```

```
##  4 Qc2   chilled     32.7 11.3      7
##  5 Qc3   chilled     32.6 10.3      7
##  6 Qc1   chilled     30.0  8.33     7
##  7 Mn2   nonchilled  27.3  7.65     7
##  8 Mn1   nonchilled  26.4  8.69     7
##  9 Mn3   nonchilled  24.1  6.48     7
## 10 Mc1   chilled     18    4.12     7
## 11 Mc3   chilled     17.3  3.05     7
## 12 Mc2   chilled     12.1  2.19     7
```

Qn3 with the nonchilled treatment has the highest mean uptake (37.61429 µmol/m²/s), and Mc2 with the chilled treatment has the lowest mean uptake (12.14286 µmol/m²/s).

c) Tidy up the data by making the data set longer, re-formatting the ambient concentration and uptake into a new column called `measurement` with the values in a new column called `values`. Call this new dataset `CO2_long`. What do you notice about the average ambient concentration, as measured for each treatment group (i.e., grouped by plant, treatment, and the measurement)? [1]

```
CO2_long <- CO2 %>%
  pivot_longer(
    cols = c("conc", "uptake"),
    names_to = "measurement",
    values_to = "values"
  )

nrow(CO2_long)
```

```
## [1] 168
```

```
CO2_long %>%
  group_by(Plant, Treatment, measurement) %>%
  summarize(
    mean = mean(values),
    .groups = "drop"
  )
```

```
## # A tibble: 24 x 4
##    Plant Treatment  measurement  mean
##    <ord> <fct>      <chr>       <dbl>
##  1 Qn1   nonchilled conc        435
##  2 Qn1   nonchilled uptake       33.2
##  3 Qn2   nonchilled conc        435
##  4 Qn2   nonchilled uptake       35.2
##  5 Qn3   nonchilled conc        435
##  6 Qn3   nonchilled uptake       37.6
##  7 Qc1   chilled    conc        435
##  8 Qc1   chilled    uptake       30.0
##  9 Qc3   chilled    conc        435
## 10 Qc3   chilled    uptake       32.6
## # i 14 more rows
```

The mean concentration is the same (435 mL/L) because all plants were tested at the same set of CO2 concentrations.

d) Using this new longer format dataset as created in c), filter cases where the treatment is nonchilled, and the measurement is uptake. Then, add a new column called `values2`, and change the uptake measurement by a multiple of 1.3. Save this into a new object called CO2_new. [1]

```
CO2_new <- CO2_long %>%
 filter(Treatment == "nonchilled" &
        measurement == "uptake") %>%
 mutate(values2 = values * 1.3)

nrow(CO2_new)
```

```
## [1] 42
```

CO2_new contains 42 rows after filtering for the nonchilled treatment and uptake measurements. The new values2 column contains the uptake values multiplied by 1.3.

e) Compare your `CO2_new` with the object below `CO2_new2`. Why are these two objects different? What does this tell you about this new function `ifelse()`? That is, how does `ifelse()` work? Hint: answer in terms of the arguments in this function. [1]

```
CO2_new2 <- CO2_long %>%
  mutate(values2 = ifelse(Treatment == "nonchilled" & measurement == "uptake",
                          values * 1.3,
                          values))

nrow(CO2_new)
```

```
## [1] 42
```

```
nrow(CO2_new2)
```

```
## [1] 168
```

```
head(CO2_new2)
```

```
## # A tibble: 6 x 6
##   Plant Type   Treatment  measurement values values2
##   <ord> <fct>  <fct>      <chr>        <dbl>   <dbl>
## 1 Qn1   Quebec nonchilled conc          95      95
## 2 Qn1   Quebec nonchilled uptake        16      20.8
## 3 Qn1   Quebec nonchilled conc         175     175
## 4 Qn1   Quebec nonchilled uptake        30.4    39.5
## 5 Qn1   Quebec nonchilled conc         250     250
## 6 Qn1   Quebec nonchilled uptake        34.8    45.2
```

The ifelse() function has three arguments: a test condition, a value returned if the condition is TRUE, and a value returned if the condition is FALSE. In this example, if Treatment is "nonchilled" and measurement is "uptake", values is multiplied by 1.3. Otherwise, the original value is kept. The filter() + mutate() approach returns only the rows that satisfy the condition, resulting in 42 rows, whereas ifelse() inside mutate() keeps all 168 rows and only modifies the matching rows.